



Département de génie informatique et génie logiciel

Cours : INF8770
Technologie Multimédia

Travail pratique 1
Compression sans perte: texte

Delan Cruz Pulgarin - 2218061

Omar Hemissi - 2219501

Groupe 01

Date de remise :
29 Septembre 2025

Partie 1 — Analyse exploratoire

Question 1 (/4)

Fichier Structuré (annual-enterprise-survey-2024-financial-year-provisional)

L'analyse effectuée sur le fichier structuré (.csv) a démontré que celui-ci présentait beaucoup de répétitions. En effet, comme on l'observe dans la figure 1, le taux de mots uniques n'est que de 1,30 %, ce qui indique un grand potentiel de compressibilité. Des mots comme *financial*, *03* et *assets* (voir figure 1) sont les plus utilisés. On remarque donc que le vocabulaire est très restreint.

Quant à l'entropie maximale possible, elle est égale à 5,9069 bits/symbole, alors que l'entropie de Shannon est inférieure à 4.6447 bits/symbole. On peut conclure que la compressibilité peut être efficace, surtout avec un taux de redondance de 21,37 %.

Ainsi, on peut formuler l'hypothèse que les méthodes de compression appliquées à ce fichier devraient produire des gains significatifs, en exploitant la fréquence élevée des répétitions et la structure limitée du vocabulaire.

Fichiers	Données	Nombre de mots	Nombre de mots uniques	Pourcentage de mots uniques	Entropie de Shannon (bits/symbole)	Entropie max. (bits/symbole)	Taux de redondance (%)
CSV		600476	7830	1.30	4.64	5.91	21.37
Texte		494	260	52,63	4.35	5.52	21.17
HTML		5749	652	11.34	4.24	6.59	35.68

Tableau 1 : Comparaison des données entre les différents textes

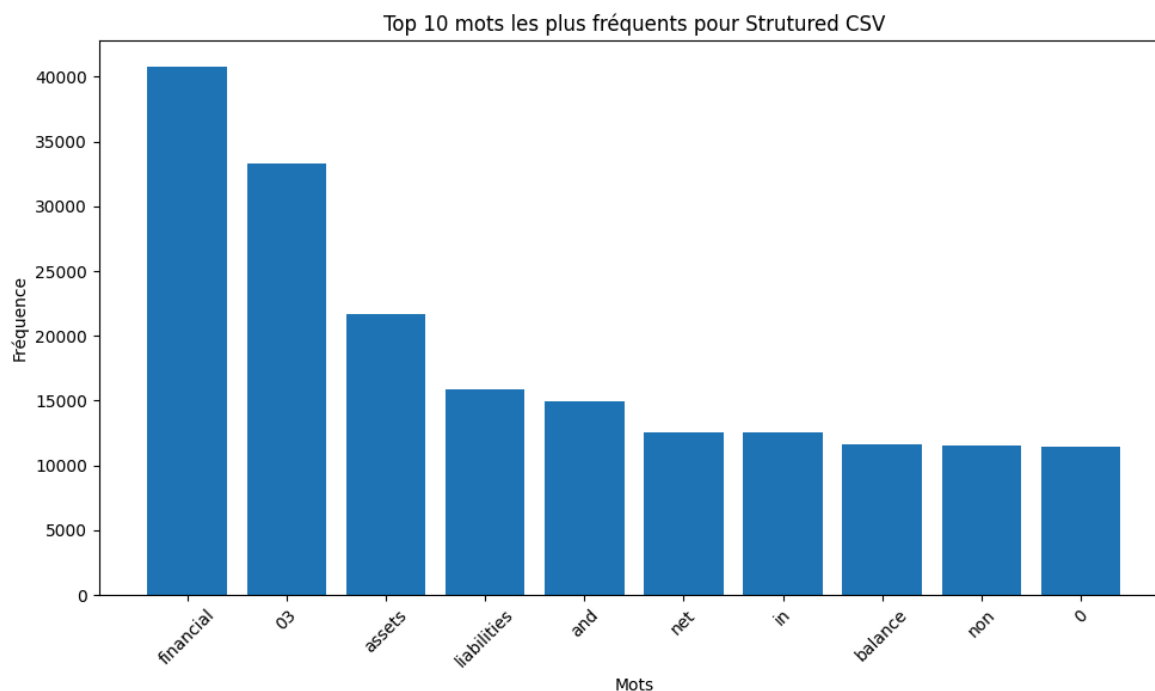


Figure 1 : Histogramme des 10 mots les plus fréquents dans le texte CSV

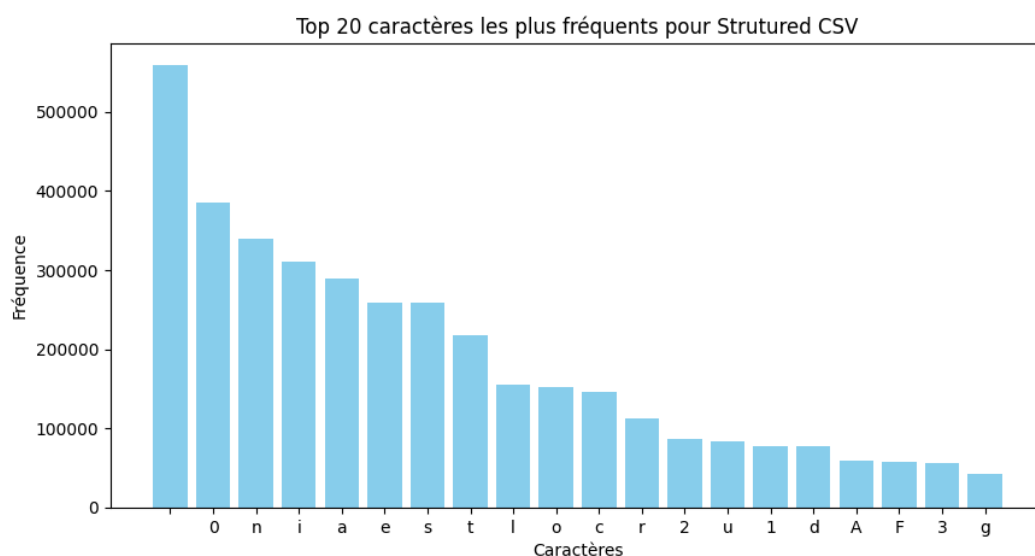


Figure 2 : Histogramme des 20 caractères les plus fréquents dans le texte CSV

Fichier texte (text_natural_1.txt)

Contrairement au fichier structuré, le premier fichier .txt a un vocabulaire très peu répétitif, puisqu'il s'agit d'une histoire courte. L'enchaînement des mots et des caractères est donc beaucoup moins structuré et prévisible. Comme on peut le voir dans la figure ci-dessous, 52,63% des mots, soit plus de la moitié, sont uniques. Cependant, les mots courants de la langue anglaise (the, of, etc) sont très utilisés, ce qui donne une certaine régularité au texte.

L'alphabet utilisé étant presque identique, l'entropie de Shannon (4.3543 bits/symbole) et l'entropie max (5.5236 bits/symbole) sont similaires aux entropies précédentes quoique quelque peu inférieures.

Il y a donc de très fortes chances que la compression entraînera des gains notables en exploitant certaines régularités dans le texte, mais la diversité lexicale plus importante et le manque de structure comparé au fichier .csv fera certainement en sorte que les gains seront moins importants.

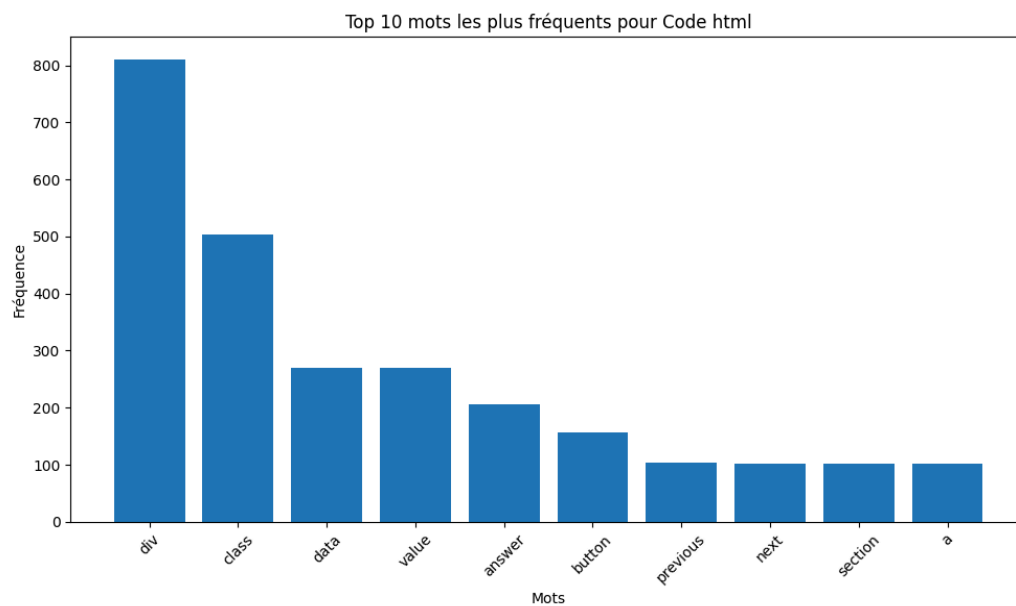


Figure 3 : Histogramme des 10 mots les plus présents dans le .txt

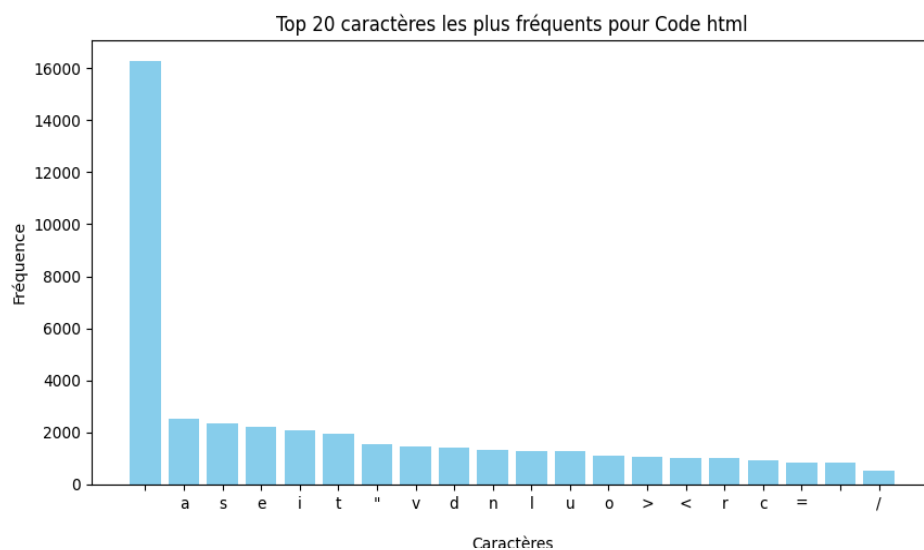


Figure 4 : Histogramme des 20 caractères les plus présents dans le .txt

Fichier Code (Code2.html)

Nous avons effectué l'analyse pour le fichier html. Ce dernier présente un bon potentiel de compressibilité. En effet, comme on le voit dans le tableau 1, seulement 11,34% des mots sont uniques. Étant donné la structure et syntaxe des fichiers html, les mots comme div et class sont souvent répétés (voir figure 5).

Quant à l'entropie maximale possible, elle est égale à 6.5850 bits/symbole, car il y a plus de symboles distincts que dans les autres fichiers (symbole spéciaux comme "<", "=" et ">"). Étant plus élevée que l'entropie de Shannon (4,2356 bits/symbole), on peut conclure que la compressibilité peut être efficace, car, surtout avec un taux de redondance de 35.68 %.

Bref, on pense que les méthodes de compression appliquées à ce fichier devraient produire des gains importants, en raison de la structure et la syntaxe du fichier html.

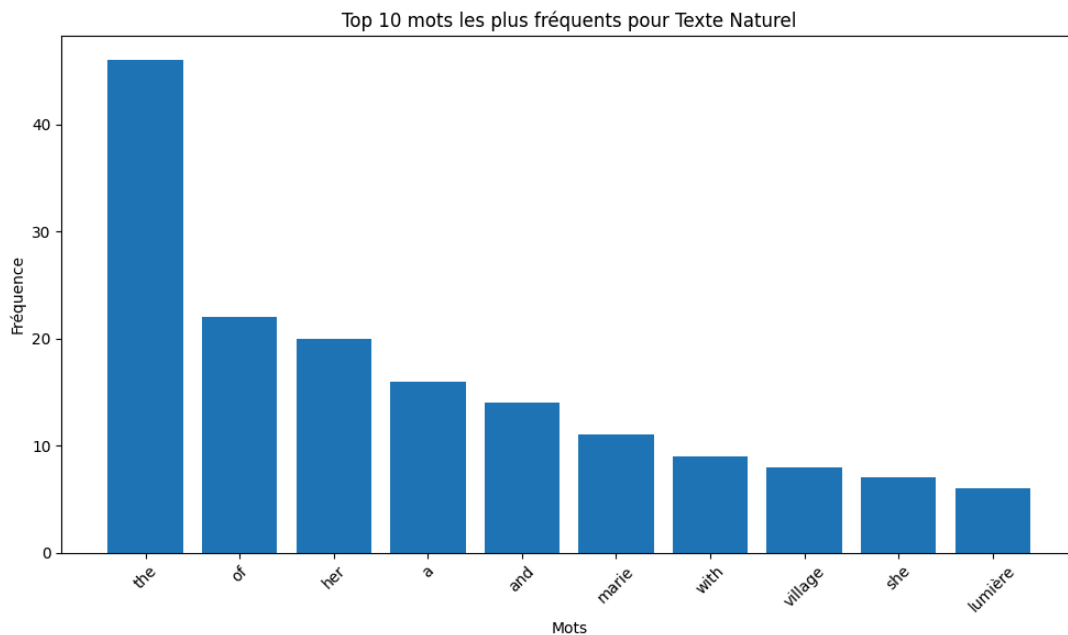


Figure 5 : Histogramme des 10 mots les plus fréquents dans le texte Code Html

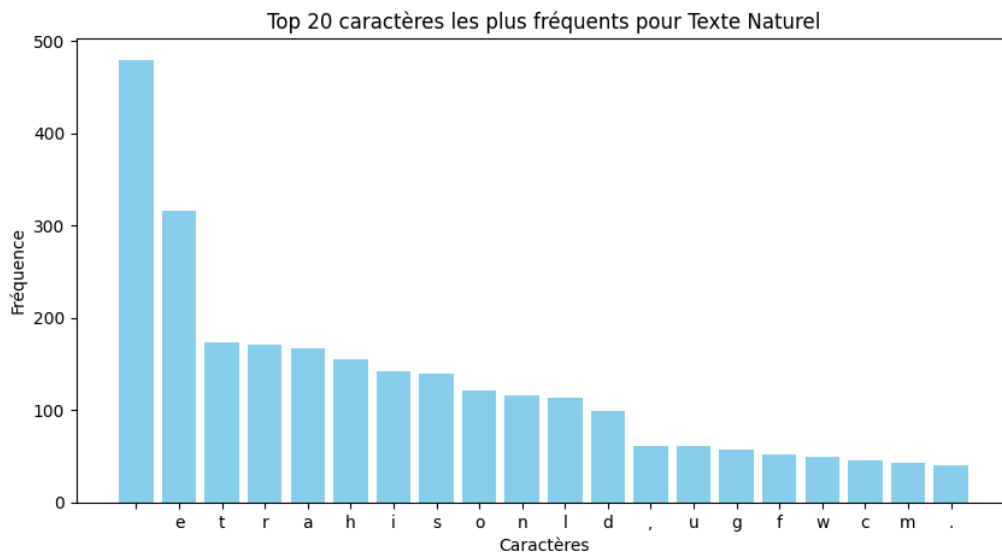


Figure 6 : Histogramme des 20 caractères les plus fréquents dans le texte Code Html

Partie 2 — Application

Question 2 (/4)

Fichier Structuré (annual-enterprise-survey-2024-financial-year-provisional)

Nous avons choisi de compresser ce fichier avec 3 méthodes différentes: Huffman, LZ77 et LZW. Huffman(codage entropique) nous permettra de réduire la taille en codant les symboles fréquents plus courts. Ensuite, on voulait utiliser du codage par dictionnaire pour exploiter le vocabulaire restreint et répétitif du fichier. LZ77 nous permettra de remplacer les séquences répétées grâce à la fenêtre glissante ce qui est bon pour de la redondance. Enfin, on teste avec LZW également, car il construit un dictionnaire dynamique des séquences récurrentes. Comparer ces trois approches nous permettra d'évaluer l'impact de la fréquence des mots et de la redondance sur la compression.

Pour ensuite évaluer et comparer les méthodes, nous allons mesurer le taux de compression ainsi que la rapidité d'exécution. Ceci nous permettra de trouver la méthode de compression la plus efficace pour ce type de fichier.

Remarque : Dans le tableau comparatif, on voit également le codage en chaîne et prédictif. Selon notre analyse, ils ne seront pas optimaux, mais ils sont présents dans les résultats à titre comparatif.

Fichier texte (text_natural_1.txt)

Les trois méthodes de compression mentionnées précédemment seront aussi utilisées pour l'histoire courte. Nous nous attendons toutefois à une diminution significative des gains engendrés par les différentes méthodes de compressions. En effet, le vocabulaire du fichier étant beaucoup plus varié et imprévisible que celui du fichier structuré, les séquences récurrentes seront moins présentes ce qui empêchera les algorithmes LZW et LZ77 d'être aussi efficaces.

Fichier Code (Code2.html)

On utilisera les mêmes méthodes. Nous nous attendons à une bonne compression, mais moins performante que le fichier structuré. En effet, le vocabulaire est modérément varié et la syntaxe et structure sont répétitives. Les algorithmes pourront donc bien exploiter ces caractéristiques pour donner de bons taux de compression.

La compression par Huffman quant à elle, devrait être assez similaire entre les différents fichiers puisqu'elle ne se base que sur la fréquence des caractères individuels. Malgré cela, elle sera sans doute moins efficace que LZW et LZ77 même si leur efficacité est amoindrie.

Question 3 (/8, 4 points par qualité)

Nous avons opté d'implémenter les algorithmes de compression en **Python** pour sa clarté et la simplicité de trouver et utiliser les librairies, qui nous ont été utiles. Nous avons utilisé **pandas** pour charger et manipuler facilement les fichiers CSV, **Counter** pour calculer la fréquence des mots, **spicy.stats.entropy** pour le calcul de l'entropie et **matplotlib** pour créer des histogrammes nous permettant de visualiser les données recueillies. Les librairies **huffman** et **gzip** ont aussi été utilisées pour l'implémentation des méthodes de compression huffman et LZ77. Finalement, **time** nous a été utile pour le minuteur qui permet de calculer le temps d'exécution.

Pour les code python, nous avons utilisé l'outil **ChatGPT** pour la correction, et implémentation de plusieurs parties de nos fichiers de code. Ceci est le cas pour le fichier [partie1.py](#), [partie2.py](#), [huffmann.py](#), [parchaine.py](#), [parplage.py](#) .

Nous avons ensuite procédé à la compression des fichiers par toutes les 4 méthodes différentes pour comparer leur taux de compression.

Fichier Structuré (annual-enterprise-survey-2024-financial-year-provisional)

Méthode de compression	Taille originale	Taille compressée	Taux de compression	Temps écoulé
LZW	4297756 caractères	194363 codes	95.48%	0.44 secondes
Huffman	34382048 bits	20767340 bits	39.6%	0.31 secondes
LZ77	4297756 octets	215421 octets	94.99%	0.07 secondes
Codage en chaîne	4297756 caractères	4034516 codes	6.13 %	0.29 secondes

Tableau 2 : Résultats de compression de 4 méthodes pour le fichier CSV

Fichier texte (text_natural_1.txt)

Méthode de compression	Taille originale	Taille compressée	Taux de compression	Temps écoulé
LZW	2845 octets	1265 octets	55.53%	0.003 sec
Huffman	22640 bits	12416 bits	45.16%	0.002 sec
LZ77	2845 octets	1380 octets	51.49%	0.001 sec
Codage en chaîne	2830 caractères	2786 codes	1.55%	0.001 sec

Tableau 3 : Résultats de compression de 4 méthodes pour le fichier texte

Fichier Code (Code2.html)

Méthode de compression	Taille originale	Taille compressée	Taux de compression	Temps écoulé
------------------------	------------------	-------------------	---------------------	--------------

LZW	49287 octets	7923 octets	83.92%	0.009 sec
Huffman	391024 bits	209337 bits	46.46%	0.004 sec
LZ77	49287 octets	5716 octets	88.40%	0.002 sec
Codage en chaîne	48878 caractères	35093 codes	28.20%	0.003 sec

Tableau 4 : Résultats de compression de 4 méthodes pour le fichier code

Question 4 (/4)

Les résultats obtenus confirment en grande partie les hypothèses formulées dans la partie 2.

- **LZW et LZ77** : Pour le fichier structuré, on a obtenu des taux de compression les plus élevés (~95 %), ce qui correspond à notre hypothèse que les méthodes basées sur la détection de séquences répétitives seraient très efficaces pour ce type de fichier. Le fichier CSV contient en effet de nombreuses répétitions et un vocabulaire limité, ce qui favorise la construction de dictionnaires dynamiques et l'exploitation de séquences répétées.

Pour le fichier texte, leur efficacité diminue grandement (55% et 51% respectivement), mais ils demeurent tout de même les plus efficaces. Cela correspond aussi à notre hypothèse, justifiable par l'imprévisibilité et le vocabulaire plus varié de son contenu.

Enfin, pour le fichier code, on a également obtenu un grand taux de compression de ~83% pour LZW et ~88% pour LZ77 (Voir tableau 4) . Il est moins élevé que le fichier structuré, car il a plus de mots uniques, mais en raison de sa structure et syntaxe utilisant des mots et symboles identiques, on réussit une bonne compression de fichier.

- **Huffman**: Il a montré un taux de compression plus faible pour le fichier structuré (~40 %). Cela était attendu, car le codage entropique agit au niveau des symboles individuels et ne capture pas efficacement les longues répétitions de chaînes présentes dans le fichier.

La compression du fichier texte donne un résultat assez similaire, quoique légèrement supérieur (45%), ce qui peut être expliqué par le fait que l'histoire ne contient pas de nombres, l'alphabet est donc plus petit. Néanmoins, il reste utile pour coder les symboles fréquents plus courts.

Comparativement aux autres fichiers, le fichier de code présente un meilleur résultat de compression (46%). Ceci est le cas, car le fichier html a beaucoup de symboles fréquents : <, >, /, =, div, class, etc. Ces symboles apparaissent très souvent, donc Huffman leur donne des codes très courts.

- **Codage en chaîne:** Il y a eu des résultats beaucoup moins bons. Le codage en chaîne ne tire pas parti des répétitions longues que l'on observe dans le fichier structuré.

Il n'est pas efficace non plus pour le texte naturel, car celui-ci n'offre aucun motif répétitif.

Quant au fichier html, le codage en chaînes a beaucoup mieux fonctionné parce qu'il contient de nombreux motifs répétitifs exacts, tel que mentionné auparavant.. Ces motifs peuvent être remplacés par des codes courts, ce qui réduit efficacement la taille.

Bref, LZW et LZ77 sont clairement les plus efficaces pour les 3 types de fichiers, car ils exploitent à la fois la redondance et la récurrence des séquences de caractères. L'algorithme LZ77 se démarque cependant par son temps d'exécution, qui est nettement inférieur aux autres méthodes, faisant de lui l'option la plus favorable. Nous voulons souligner que les temps d'exécution étaient relativement les mêmes pour tous les algorithmes. Donc nous considérons que ça n'impacte pas le choix du meilleur algorithme de compression.

Bibliographie

[1] Bilodeau, Guillaume-Alexandre. "Chapitre 2 - Compression sans perte". Polytechnique Montréal. [En ligne]. Disponible: https://moodle.polymtl.ca/pluginfile.php/1386019/mod_resource/content/30/Chap2_CompressionSansPerte.pdf . [Consulté en septembre 2025].

[2] OpenAI, "ChatGPT (version GPT-5)," OpenAI, San Francisco, CA, USA. [En ligne]. Disponible: <https://chat.openai.com/>. [Consulté en septembre 2025].